```
import os
import json
import requests
import pandas as pd
from datetime import datetime

# ────────────────────────────────────────────
# 配置
# ────────────────────────────────────────────
TELEGRAM_TOKEN   = os.environ["TELEGRAM_TOKEN"]
TELEGRAM_CHAT_ID = os.environ["TELEGRAM_CHAT_ID"]

SYMBOL   = "BTCUSDT"
INTERVAL = "1h"
LIMIT    = 250

# ATR 参数
ATR_PERIOD        = 14
ATR_STOP_MULT     = 2.0   # 初始止损 = 入场价 - ATR × 2.0
ATR_TRAIL_MULT    = 1.5   # 移动止损距离 = ATR × 1.5
TRAIL_ACTIVATE_PCT = 0.015  # 盈利达到 +1.5% 后激活移动止损
TAKE_PROFIT_MULT  = 4.0   # 止盈目标 = 入场价 + ATR × 4.0

# 止损/止盈的安全上下限（防止 ATR 异常时设出离谱的值）
STOP_MIN_PCT  = 0.01   # 止损最小 -1%
STOP_MAX_PCT  = 0.04   # 止损最大 -4%
TARGET_MIN_PCT = 0.02  # 止盈最小 +2%
TARGET_MAX_PCT = 0.06  # 止盈最大 +6%


# ────────────────────────────────────────────
# 1. K 线
# ────────────────────────────────────────────
def get_klines(symbol, interval, limit):
    url = "https://api.binance.com/api/v3/klines"
    r = requests.get(url, params={"symbol": symbol, "interval": interval, "limit"
    r.raise_for_status()
    df = pd.DataFrame(r.json(), columns=[
        "open_time","open","high","low","close","volume",
        "close_time","quote_volume","trades",
        "taker_buy_base","taker_buy_quote","ignore"
    ])
    for col in ["open","high","low","close","volume"]:
        df[col] = df[col].astype(float)
```

```
    return df


# ────────────────────────────────────────
# 2. 合约情绪
# ────────────────────────────────────────
def get_funding_rate():
    try:
        r = requests.get("https://fapi.binance.com/fapi/v1/premiumIndex",
                         params={"symbol": SYMBOL}, timeout=10)
        r.raise_for_status()
        return float(r.json()["lastFundingRate"])
    except Exception as e:
        print(f"资金费率获取失败: {e}")
        return None

def get_long_short_ratio():
    try:
        r = requests.get("https://fapi.binance.com/futures/data/globalLongShortAc
                         params={"symbol": SYMBOL, "period": "1h", "limit": 1}, t
        r.raise_for_status()
        data = r.json()
        return float(data[0]["longShortRatio"]) if data else None
    except Exception as e:
        print(f"多空比获取失败: {e}")
        return None

def get_open_interest_change():
    try:
        r = requests.get("https://fapi.binance.com/futures/data/openInterestHist"
                         params={"symbol": SYMBOL, "period": "1h", "limit": 5}, t
        r.raise_for_status()
        data = r.json()
        if len(data) >= 2:
            latest = float(data[-1]["sumOpenInterestValue"])
            oldest = float(data[0]["sumOpenInterestValue"])
            return round((latest - oldest) / oldest * 100, 2), round(latest / 1e8
        return None, None
    except Exception as e:
        print(f"持仓量获取失败: {e}")
        return None, None


# ────────────────────────────────────────
# 3. 技术指标（含 ATR）
# ────────────────────────────────────────
def compute_indicators(df):
```

```
    # 趋势
    df["ma200"] = df["close"].rolling(200).mean()
    df["ma50"]  = df["close"].rolling(50).mean()

    # RSI
    delta = df["close"].diff()
    gain  = delta.clip(lower=0).rolling(14).mean()
    loss  = (-delta.clip(upper=0)).rolling(14).mean()
    df["rsi"] = 100 - (100 / (1 + gain / loss))

    # 成交量
    df["vol_ma20"] = df["volume"].rolling(20).mean()

    # 近期结构
    df["recent_high_40"] = df["high"].rolling(40).max()
    df["recent_low_40"]  = df["low"].rolling(40).min()

    # ── ATR（True Range 的移动平均）──────
    df["prev_close"] = df["close"].shift(1)
    df["tr"] = pd.concat([
        df["high"] - df["low"],                          # 当根高低差
        (df["high"] - df["prev_close"]).abs(),           # 高点与前收盘差
        (df["low"]  - df["prev_close"]).abs(),           # 低点与前收盘差
    ], axis=1).max(axis=1)
    df["atr"] = df["tr"].rolling(ATR_PERIOD).mean()

    return df


# ────────────────────────────────────────────
# 4. 动态止损 / 止盈计算
# ────────────────────────────────────────────
def calc_dynamic_levels(entry_price, atr):
    """根据入场价和当时 ATR 计算初始止损和止盈"""
    raw_stop   = entry_price - atr * ATR_STOP_MULT
    raw_target = entry_price + atr * TAKE_PROFIT_MULT

    # 安全上下限
    stop = max(raw_stop,   entry_price * (1 - STOP_MAX_PCT))
    stop = min(stop,       entry_price * (1 - STOP_MIN_PCT))
    target = max(raw_target, entry_price * (1 + TARGET_MIN_PCT))
    target = min(target,     entry_price * (1 + TARGET_MAX_PCT))

    stop_pct   = (stop   / entry_price - 1) * 100
    target_pct = (target / entry_price - 1) * 100

    return round(stop, 1), round(target, 1), round(stop_pct, 2), round(target_pct
```

```
def calc_trailing_stop(highest_price, atr):
    """移动止损位 = 最高价 - ATR × 1.5"""
    return round(highest_price - atr * ATR_TRAIL_MULT, 1)


# ────────────────────────────────────────────
# 5. 信号判断
# ────────────────────────────────────────────
def check_signal(df, funding_rate, ls_ratio):
    last  = df.iloc[-1]
    prev3 = df.iloc[-4]

    price       = last["close"]
    ma200       = last["ma200"]
    ma50        = last["ma50"]
    rsi_now     = last["rsi"]
    rsi_3ago    = prev3["rsi"]
    vol_now     = last["volume"]
    vol_ma      = last["vol_ma20"]
    recent_high = last["recent_high_40"]
    recent_low  = last["recent_low_40"]
    atr         = last["atr"]

    # ══ 加权评分系统 ═══════════════════
    #
    # 权重设计原则:
    #   趋势是前提（最高权重），动量次之，辅助条件补充
    #   满分 11.0，入场阈值 6.5
    #   部分满足给半分，避免"差一个条件全军覆没"
    #
    score  = 0.0
    scores = {}  # 用于消息展示

    # ── 1. 中期趋势：价格在 MA200 之上 ──────────── 权重 3.0
    # 这是硬性前提，不满足直接不参与评分（后面硬过滤）
    if price > ma200:
        s = 3.0
    else:
        s = 0.0
    score += s
    scores["趋势MA200"] = (s, 3.0, price > ma200)

    # ── 2. 短期趋势：MA50 向上倾斜 ──────────────── 权重 1.5
    ma50_now  = last["ma50"]
    ma50_4ago = df.iloc[-4]["ma50"]
```

```
if ma50_now > ma50_4ago:
    s = 1.5                              # 明确向上
elif ma50_now > df.iloc[-8]["ma50"]:
    s = 0.5                              # 8根前仍在上升（趋势减速但未反转）
else:
    s = 0.0
score += s
scores["MA50斜率"] = (s, 1.5, s > 0)

# ── 3. RSI 动量：从超卖区回升 ──────────────── 权重 2.5
if rsi_3ago < 38 and rsi_now > 44:
    s = 2.5                              # 强回升：超卖后明确反弹
elif rsi_3ago < 45 and rsi_now > 44:
    s = 1.0                              # 弱回升：未到超卖但有上升动能
elif rsi_now > 50 and rsi_now < 65:
    s = 0.5                              # RSI 健康区间，不拖累分数
else:
    s = 0.0
score += s
scores["RSI动量"] = (s, 2.5, s >= 1.0)

# ── 4. 成交量确认 ──────────────────────────── 权重 2.0
vol_ratio = vol_now / vol_ma
if vol_ratio >= 1.3:
    s = 2.0                              # 明显放量
elif vol_ratio >= 1.1:
    s = 1.0                              # 温和放量
else:
    s = 0.0
score += s
scores["成交量"] = (s, 2.0, s >= 1.0)

# ── 5. 位置合理：不追高 ────────────────────── 权重 1.5
dist_from_high = (price / recent_high - 1) * 100  # 负值=低于高点
if dist_from_high < -5:
    s = 1.5                              # 距高点 5% 以上，位置好
elif dist_from_high < -3:
    s = 0.8                              # 距高点 3-5%，可接受
else:
    s = 0.0                              # 距高点 3% 以内，追高风险高
score += s
scores["位置"] = (s, 1.5, s > 0)

# ── 6. 接近支撑（加分项）──────────────────── 权重 0.5
dist_from_low = (price / recent_low - 1) * 100    # 正值=高于低点
if dist_from_low < 4:
    s = 0.5                              # 距近期低点很近，有支撑
```

```
else:
    s = 0.0
score += s
scores["支撑位"] = (s, 0.5, s > 0)

score = round(score, 1)

# ── 信号强度等级 ──────────────────────────────
if score >= 9.5:
    strength      = "⭐⭐⭐⭐⭐ 极强"
    strength_short = "极强"
elif score >= 8.0:
    strength      = "⭐⭐⭐⭐ 较强"
    strength_short = "较强"
elif score >= 6.5:
    strength      = "⭐⭐⭐ 一般"
    strength_short = "一般"
else:
    strength      = "⭐⭐ 弱"
    strength_short = "弱"

# ═══ 硬性过滤（不参与评分，直接阻断）═══════════
# 1. 必须在 MA200 之上（下跌趋势中不做多）
trend_ok = price > ma200

# 2. 资金费率过高
funding_ok           = True
funding_block_reason = None
if funding_rate is not None and funding_rate > 0.0008:
    funding_ok           = False
    funding_block_reason = f"资金费率过高 ({funding_rate*100:.3f}%)，多头拥挤"

ENTRY_THRESHOLD = 6.5
tech_signal  = trend_ok and (score >= ENTRY_THRESHOLD)
entry_signal = tech_signal and funding_ok

ls_warning = ls_ratio is not None and ls_ratio > 1.6

return {
    "price":        price,
    "ma200":        round(ma200, 1),
    "ma50":         round(ma50, 1),
    "rsi":          round(rsi_now, 1),
    "rsi_3ago":     round(rsi_3ago, 1),
    "vol_ratio":    round(vol_ratio, 2),
    "recent_high":  round(recent_high, 1),
    "atr":          round(atr, 1),
```

```
        "atr_pct":      round(atr / price * 100, 3),
        "score":        score,
        "score_max":    11.0,
        "threshold":    ENTRY_THRESHOLD,
        "strength":     strength,
        "strength_short": strength_short,
        "scores":       scores,       # {条件: (得分, 满分, 是否触发)}
        "entry_signal": entry_signal,
        "tech_signal":  tech_signal,
        "funding_ok":   funding_ok,
        "funding_block_reason": funding_block_reason,
        "ls_warning":   ls_warning,
        "trend_broken": price < ma200,
        "ma50_broken":  price < ma50,
    }


# ────────────────────────────────────────────
# 6. 持仓状态 (JSON 格式)
# ────────────────────────────────────────────
STATE_FILE = "state.txt"

def read_state():
    """
    返回 dict 或 None (空仓)
    字段: entry_price, initial_stop, take_profit, atr,
          highest, trailing_stop, trailing_active
    """
    if os.path.exists(STATE_FILE):
        content = open(STATE_FILE).read().strip()
        if content and content != "NONE":
            try:
                return json.loads(content)
            except Exception:
                # 兼容旧版纯数字格式
                try:
                    ep = float(content)
                    return {"entry_price": ep, "initial_stop": None,
                            "take_profit": None, "atr": None,
                            "highest": ep, "trailing_stop": None,
                            "trailing_active": False}
                except Exception:
                    pass
    return None

def write_state(state_dict):
    with open(STATE_FILE, "w") as f:
```

```
        if state_dict is None:
            f.write("NONE")
        else:
            json.dump(state_dict, f)


# ────────────────────────────────────────────
# 7. 情绪解读
# ────────────────────────────────────────────
def interpret_funding(rate):
    if rate is None: return "获取失败", "neutral"
    pct = rate * 100
    if   pct >  0.08: return f"+{pct:.3f}% 🔥 多头极度拥挤", "overheated"
    elif pct >  0.03: return f"+{pct:.3f}% ⚠️ 多头偏热",     "warm"
    elif pct > -0.01: return f"{pct:+.3f}% ✅ 中性",          "neutral"
    elif pct > -0.05: return f"{pct:+.3f}% 🧊 空头偏多",      "cool"
    else:             return f"{pct:+.3f}% ❄️ 空头极度拥挤",  "cold"

def interpret_ls(ratio):
    if ratio is None: return "获取失败"
    if   ratio > 1.5: return f"{ratio:.2f} ⚠️ 散户严重偏多（反向信号）"
    elif ratio > 1.1: return f"{ratio:.2f} 多头略占优"
    elif ratio > 0.9: return f"{ratio:.2f} ✅ 多空均衡"
    elif ratio > 0.7: return f"{ratio:.2f} 空头略占优"
    else:             return f"{ratio:.2f} ⚠️ 散户严重偏空（潜在反弹）"

def interpret_oi(change_pct):
    if change_pct is None: return "获取失败"
    if   change_pct >  3: return f"+{change_pct}% 📈 资金大量流入"
    elif change_pct >  0: return f"+{change_pct}% 资金小幅流入"
    elif change_pct > -3: return f"{change_pct}% 资金小幅撤离"
    else:                 return f"{change_pct}% 📉 资金大量撤离"


# ────────────────────────────────────────────
# 8. Telegram
# ────────────────────────────────────────────
def send_telegram(msg):
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
    r = requests.post(url, json={
        "chat_id": TELEGRAM_CHAT_ID,
        "text": msg,
        "parse_mode": "HTML"
    }, timeout=10)
    r.raise_for_status()
    print("✅ Telegram 已发送")
```

```
# ────────────────────────────────────────────
# 9. 主流程
# ────────────────────────────────────────────
def main():
    now_utc   = datetime.utcnow()
    now_str   = now_utc.strftime("%Y-%m-%d %H:%M UTC")
    now_cst_h = (now_utc.hour + 8) % 24

    print(f"[{now_str}] 开始检查...")

    df             = get_klines(SYMBOL, INTERVAL, LIMIT)
    df             = compute_indicators(df)
    funding_rate   = get_funding_rate()
    ls_ratio       = get_long_short_ratio()
    oi_change, oi_val = get_open_interest_change()

    sig   = check_signal(df, funding_rate, ls_ratio)
    price = sig["price"]
    atr   = sig["atr"]
    state = read_state()

    funding_text, _ = interpret_funding(funding_rate)
    ls_text         = interpret_ls(ls_ratio)
    oi_text         = interpret_oi(oi_change)

    sentiment_block = (
        f"\n📈 <b>市场情绪</b>\n"
        f"  资金费率: {funding_text}\n"
        f"  多空比: {ls_text}\n"
        f"  持仓量(4H): {oi_text}"
        + (f" ({oi_val}亿U) " if oi_val else "")
    )

    print(f"价格:{price:.1f} ATR:{atr:.1f}({sig['atr_pct']}%) RSI:{sig['rsi']} 量比
    print(f"评分:{sig['score']}/{sig['score_max']} 阈值:{sig['threshold']} 强度:{si
    print(f"分项:{ {k: f\"{v[0]}/{v[1]}\" for k,v in sig['scores'].items()} }")
    print(f"持仓:{ json.dumps(state) if state else '空仓' }")

    # ═══ 空仓: 检查入场 ═══════════════════
    if state is None:

        if sig["entry_signal"]:
            stop, target, stop_pct, target_pct = calc_dynamic_levels(price, atr)
            rr = round(abs(target_pct / stop_pct), 1)

            dist_ma200 = round((price / sig["ma200"] - 1) * 100, 1)
```

```
        dist_high  = round((price / sig["recent_high"] - 1) * 100, 1)
        ls_warn    = "\n  ⚠️ 多空比偏高，注意散户情绪风险" if sig["ls_warning"] e

        # 评分明细
        score_lines = "\n".join([
            f"  {'✅' if v[2] else '🔶'} {k}: {v[0]}/{v[1]}"
            for k, v in sig["scores"].items()
        ])

        new_state = {
            "entry_price":    price,
            "initial_stop":   stop,
            "take_profit":    target,
            "atr":            atr,
            "highest":        price,
            "trailing_stop":  None,
            "trailing_active": False
        }
        write_state(new_state)

        msg = (
            f"🟢 <b>BTC 入场提醒</b>\n"
            f"{'—' * 22}\n"
            f"🕐 {now_str} (北京 {now_cst_h:02d}:xx) \n\n"
            f"💰 当前价: <b>${price:,.1f}</b>\n\n"
            f"🎯 <b>信号评分: {sig['score']}/{sig['score_max']}  {sig['strengt
            f"{score_lines}\n\n"
            f"📐 <b>ATR 动态止损</b> (ATR={atr:.1f} / {sig['atr_pct']}%) \n"
            f"  初始止损: <b>${stop:,}</b> ({stop_pct:.2f}%) \n"
            f"  止盈目标: <b>${target:,}</b> (+{target_pct:.2f}%) \n"
            f"  盈亏比: 1 : {rr}\n"
            f"  移动止损: 盈利 +1.5% 后激活\n"
            f"{sentiment_block}"
            f"{ls_warn}\n\n"
            f"💡 建议仓位: 账户 30%\n"
            f"⚠️ 仅供参考，请自行判断"
        )
        send_telegram(msg)

    elif sig["tech_signal"] and not sig["funding_ok"]:
        msg = (
            f"🟡 <b>BTC 信号被情绪过滤</b>\n"
            f"{'—' * 22}\n"
            f"🕐 {now_str}\n\n"
            f"技术条件: ✅ 全部满足\n"
            f"当前价: ${price:,.1f}\n\n"
            f"🚫 <b>过滤原因</b>\n"
```

```
            f"  {sig['funding_block_reason']}\n"
            f"{sentiment_block}\n\n"
            f"等待资金费率回落至 +0.05% 以下"
        )
        send_telegram(msg)

    else:
        if now_cst_h == 9:
            score_lines = "\n".join([
                f"  {'✅' if v[2] else '🔶'} {k}: {v[0]}/{v[1]}"
                for k, v in sig["scores"].items()
            ])
            msg = (
                f"📡 <b>BTC 系统运行正常</b>\n"
                f"{'—' * 22}\n"
                f"🕐 {now_str}\n\n"
                f"状态: 空仓等待\n"
                f"当前价: ${price:,.1f}\n"
                f"ATR: {atr:.1f} ({sig['atr_pct']}%) \n\n"
                f"<b>当前评分: {sig['score']}/{sig['score_max']}  {sig['strengt
                f"入场阈值: {sig['threshold']} 分\n"
                f"{score_lines}"
                f"{sentiment_block}"
            )
            send_telegram(msg)
        else:
            print("无信号, 静默")

# ═══ 持仓: 更新移动止损 + 检查出场 ═══
else:
    entry_price    = state["entry_price"]
    initial_stop   = state["initial_stop"] or entry_price * (1 - STOP_MIN_PCT
    take_profit    = state["take_profit"]  or entry_price * (1 + TARGET_MIN_P
    highest        = max(state.get("highest", entry_price), price)
    trailing_stop  = state.get("trailing_stop")
    trailing_active = state.get("trailing_active", False)
    current_atr    = state.get("atr", atr)  # 用入场时的ATR, 保持止损基准稳定

    pnl_pct = (price - entry_price) / entry_price * 100

    # ── 移动止损逻辑 ──────────────────────
    if not trailing_active and pnl_pct >= TRAIL_ACTIVATE_PCT * 100:
        # 盈利达到 +1.5%, 激活移动止损
        trailing_active = True
        trailing_stop   = calc_trailing_stop(highest, current_atr)
        print(f"✅ 移动止损激活: 最高价 {highest:.1f}, 移动止损 {trailing_stop:.1f
```

```
elif trailing_active:
    # 已激活：随价格上涨更新移动止损（只能上移，不能下移）
    new_trail = calc_trailing_stop(highest, current_atr)
    if trailing_stop is None or new_trail > trailing_stop:
        trailing_stop = new_trail

# 更新状态
state["highest"]         = highest
state["trailing_stop"]   = trailing_stop
state["trailing_active"] = trailing_active
write_state(state)

# ── 出场判断 ──────────────────────────
exit_reason = None
exit_emoji  = "🔴"

if price >= take_profit:
    exit_reason = f"达到止盈目标 +{(take_profit/entry_price-1)*100:.1f}%"
    exit_emoji  = "🎯"
elif trailing_active and trailing_stop and price <= trailing_stop:
    locked_pct = (trailing_stop / entry_price - 1) * 100
    exit_reason = f"移动止损触发（锁住 {locked_pct:+.2f}%）"
    exit_emoji  = "🔒"
elif price <= initial_stop:
    exit_reason = f"触及初始止损（ATR×{ATR_STOP_MULT}）"
    exit_emoji  = "🛡️"
elif sig["trend_broken"]:
    exit_reason = "价格跌破 MA200，趋势破坏"
elif sig["ma50_broken"] and pnl_pct < 0:
    exit_reason = "跌破 MA50 且浮亏，建议减仓"

if exit_reason:
    msg = (
        f"{exit_emoji} <b>BTC 出场提醒</b>\n"
        f"{'─' * 22}\n"
        f"🕐 {now_str}\n\n"
        f"当前价：<b>${price:,.1f}</b>\n"
        f"入场价：${entry_price:,.1f}\n"
        f"最高价：${highest:,.1f}\n"
        f"盈亏：<b>{pnl_pct:+.2f}%</b>\n\n"
        f"触发：{exit_reason}\n"
        f"{sentiment_block}\n\n"
        f"⚠️ 请自行决定是否出场"
    )
    send_telegram(msg)
    write_state(None)
```

```
        else:
            # 每4小时发持仓更新
            if now_utc.hour % 4 == 0:
                trail_line = ""
                if trailing_active and trailing_stop:
                    trail_pct  = round((trailing_stop / entry_price - 1) * 100, 2
                    dist_trail = round((price / trailing_stop - 1) * 100, 1)
                    trail_line = (
                        f"\n🔒 移动止损: <b>${trailing_stop:,}</b> ({trail_pct:+.2f
                        f"距当前 -{dist_trail}%) \n"
                        f"  最高价: ${highest:,.1f}"
                    )
                else:
                    activate_gap = round(TRAIL_ACTIVATE_PCT * 100 - pnl_pct, 2)
                    trail_line = f"\n⏳ 移动止损: 未激活 (再涨 {activate_gap:.2f}% 激

                dist_target = round((take_profit / price - 1) * 100, 1)
                dist_stop   = round((price / initial_stop - 1) * 100, 1)

                msg = (
                    f"📋 <b>BTC 持仓更新</b>\n"
                    f"{'—' * 22}\n"
                    f"🕐 {now_str}\n\n"
                    f"入场价: ${entry_price:,.1f}\n"
                    f"当前价: ${price:,.1f}\n"
                    f"浮动盈亏: <b>{pnl_pct:+.2f}%</b>\n\n"
                    f"止盈目标: ${take_profit:,.1f} (还差 +{dist_target}%) \n"
                    f"初始止损: ${initial_stop:,.1f} (距当前 -{dist_stop}%) "
                    f"{trail_line}"
                    f"{sentiment_block}"
                )
                send_telegram(msg)
                print("📋 持仓更新已发送")
            else:
                trail_status = f"移动止损@{trailing_stop:.1f}" if trailing_active e
                print(f"持仓中 {pnl_pct:+.2f}% | {trail_status} | 静默")


if __name__ == "__main__":
    main()
```